



PSOC™ Edge: Encryption

Customer training workshop (CTW)

May 2026



Prerequisites

Training prerequisites

- Introduction to PSOC™ Edge and ModusToolbox™
- For an introduction to PSOC™ MCUs, including a getting started guide to ModusToolbox™, visit: <https://www.infineon.com/product-information/psocdeveloper>
- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, visit: [PSOC™ Edge E84 Training Collection](#)

Development tools

- ModusToolbox™ v3.7
- Eclipse IDE for ModusToolbox™ v2026.1.0
- Edge Protect Security Suite v1.6
- ModusToolbox™ Programming Tools v1.7.0
- KIT_PSE84_EVAL_EPC2 BSP v1.0.0
- Terminal Emulator
- KIT_PSE84_EVAL

Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Introduction

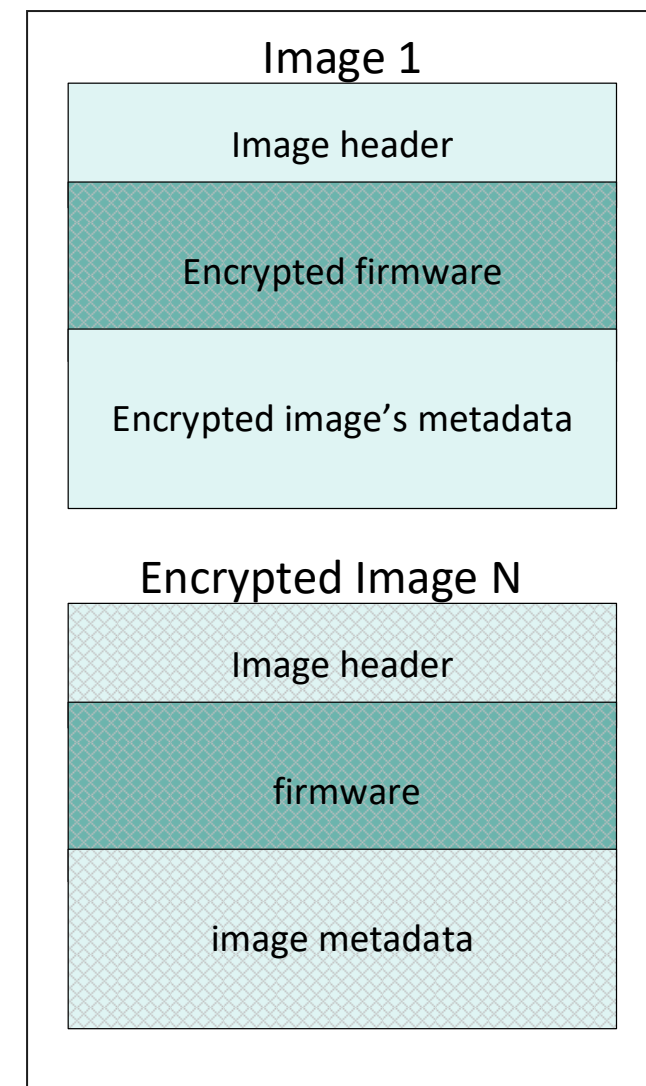
- Firmware image can be encrypted and programmed into device memory
- Edge Protect Bootloader (EPB) supports booting encrypted firmware only when running from RRAM
- Encrypted user images must reside in external flash and either executed in SMIF XIP mode (for EPC 2) or loaded to SRAM for execution (for EPC 4)
- Edge Protect Bootloader supports encrypted application execution via:
 - Single-key encryption (EPC 2)
 - Multi-key encryption (EPC 2)
 - Secure encryption (EPC 4)

Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Single-key XIP encryption: EPC 2

- All three application images use the **Advanced Encryption Standard key (AES)**
- The first image contains the AES key and AES nonce (part of the initial vector), packed in an HMAC-based Key Derivation Function (HKDF) block and stored in a Type-Length-Value field (TLV)
- For the first image, only the payload (application code) is encrypted; the header and TLVs remain in plain text
- The second and subsequent images are fully encrypted (including header and TLVs) using the same AES key and nonce as the first image



Single-key XIP encryption: EPC 2 (contd.)

- An Elliptic Curve Cryptography (ECC) key pair, referred to as the Key Encryption Key (KEK), is used to encrypt the encryption key. The private key of the KEK must be stored at a location accessible to the EPB. By default, this location is in RRAM at address 0x3203A000
- Supported with only the overwrite upgrade mode
- See [Encrypted Boot](#) code example for enabling single-key encryption

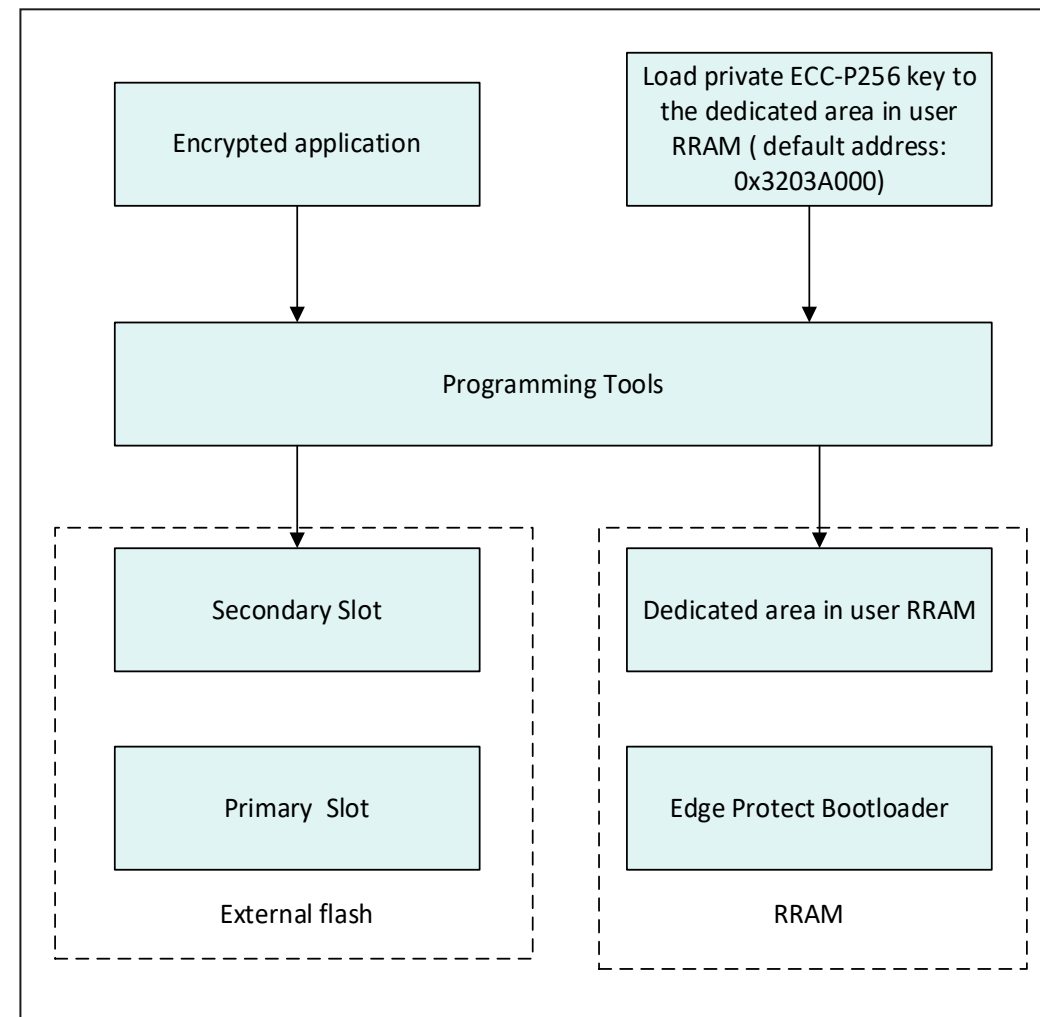
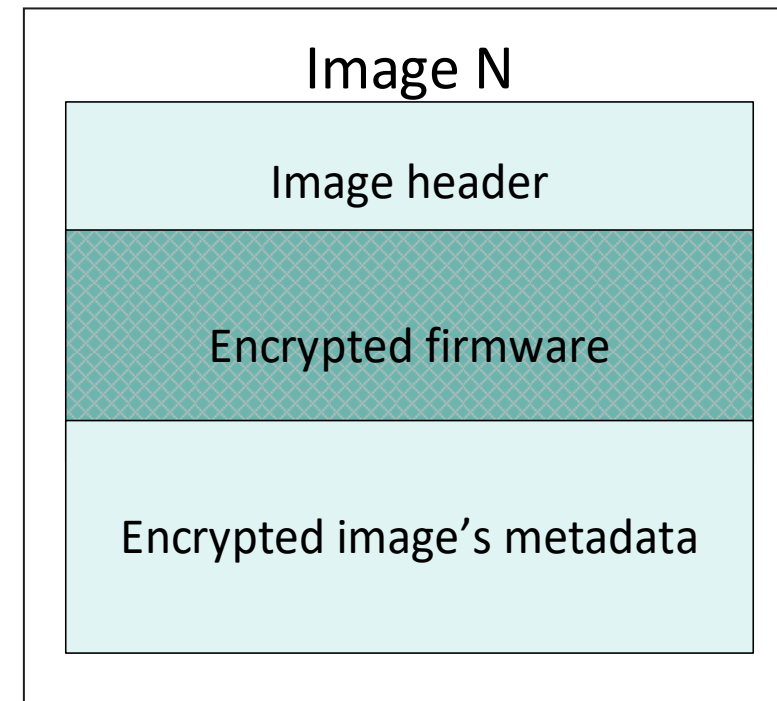


Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Multi-key XIP encryption: EPC 2

- Each image always contains its own AES key and AES nonce (part of the initial vector) packed to the HKDF block and stored in one of the TLVs in the image
- Up to four different encrypted areas and different AES keys are supported
- Only the image payload (application code part) is encrypted; the header and image TLV are stored as plain data
- Multi-key XIP encryption has strict requirements for encryption region size and start address. The slot size of the application must be a perfect power-of-two number. The start address of the application image must be a multiple of its slot size, i.e., $\text{start_address} = N * \text{size}$



Multi-key XIP encryption: EPC 2 (contd.)

- An ECC key pair (Key-Encryption-Key) is used to encrypt the encryption key; the private portion of this Key-Encryption-Key must be stored at a location known to the EPB. The default location is in the RRAM (0x3203A000)
- Supported with swap and overwrite upgrade mode
- See [Edge Protect Bootloader](#) Application Note for more details

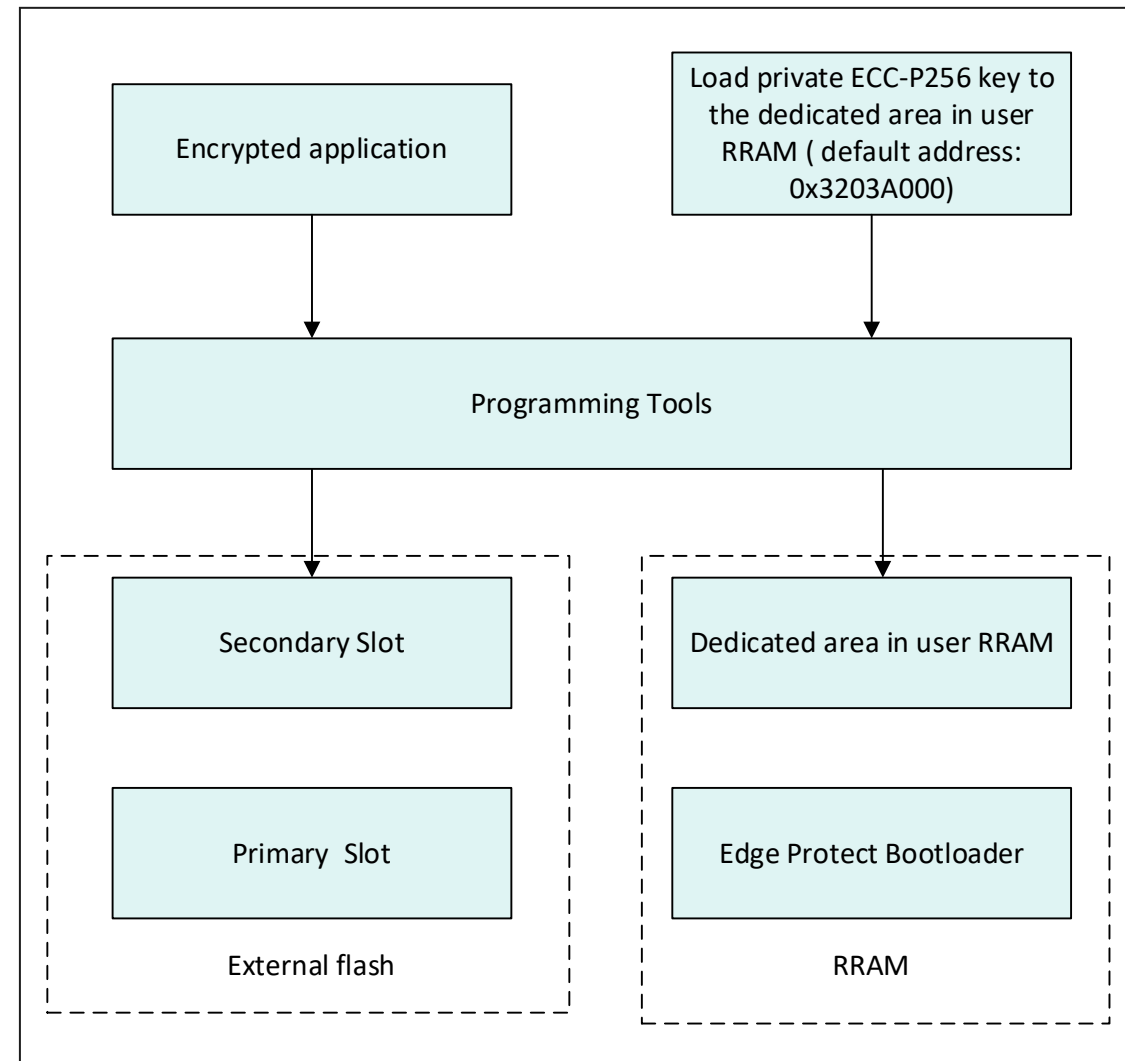


Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Secure encryption: EPC 4

- On EPC 4 devices, a software-key-based encryption mechanism is supported, compliant with PSA L4 certification
- For this use case, the private portion of the KEK must be provisioned into the device using the “user keys” section in the OEM policy file
- This method uses the AES CMAC algorithm for encryption
- This flow can only be used when the images are loaded into SRAM and executed from there
- See [Edge Protect Bootloader](#) Application Note for more details

Sample user keys policy for provisioning the Key-encryption-key (KEK)

```
{
  "kid": {
    "description": "The ID of the key in the PSA vault. The value between '0x40000000'
and '0x7FFFFFFF'",
    "value": "key_id"
  },
  "alg": {
    "description": "Key algorithm",
    "applicable_conf": "ECDSA, RSA, AES-CTR, AES-CBC, AES-ECB, ECDH, HKDF_ALG_SHA_256,
NONE",
    "value": "ECDH"
  },
  "use": {
    "description": "Key usage flags are encoded in a bitmask (usage example SIGN|
ENCRYPT)",
    "applicable_conf": "EXPORT, COPY, CACHE, ENCRYPT, DECRYPT, SIGN_MESSAGE,
VERIFY_MESSAGE, SIGN_HASH, VERIFY_HASH, DERIVE, VERIFY_DERIVATION",
    "value": "DERIVE"
  },
  "key": {
    "description": "Path to the key file in PEM, DER, or JWK format",
    "value": "../keys/enc-ec256-priv.pem"
  },
  "owner": {
    "description": "Key owner identifier. The value between '-2147483648' and
'2147483647'",
    "value": 0
  }
}
```

Table of contents

1	Introduction	4
2	Single-key XIP encryption	6
3	Multi-key XIP encryption	9
4	Secure encryption	12
5	Lab	15

Lab 1: Single-key XIP encryption with EPB

What will you learn?

- How to create the Encrypted Boot code example
- How to add a bootloader to the code example and enable the secure boot
- How to program and boot to the encrypted application

Lab 2: Multi-key XIP encryption

What will you learn?

- How to program and boot to the application using the multi-key encryption

Revision history

Document revision	Date	Description of change
**	2026-01-14	Initial release.
*A	2026-05-04	Added speaker notes. Updated version of tools. Added slides for labs.

